

Distributed Monotone-Policy Encryption for DNFs from Lattices

Jeffrey Champion and David Wu

Threshold Encryption

[DesmedtFrankel89]



master public key



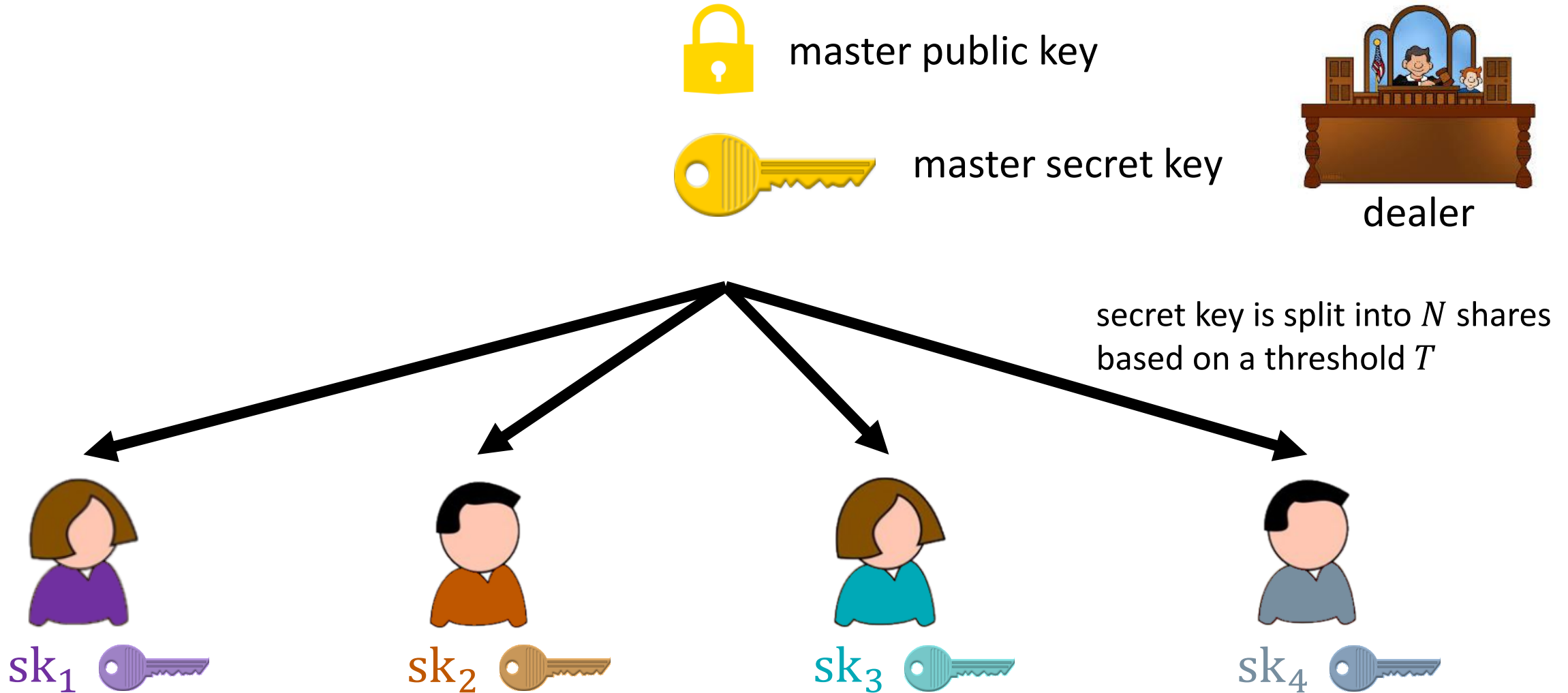
master secret key



dealer

Threshold Encryption

[DesmedtFrankel89]



Threshold Encryption

[DesmedtFrankel89]

Functionality: Sets of T users can decrypt

Security: Sets of at most $T - 1$ users learn nothing

Efficiency: $|ct| = \text{poly}(\lambda, \log N)$



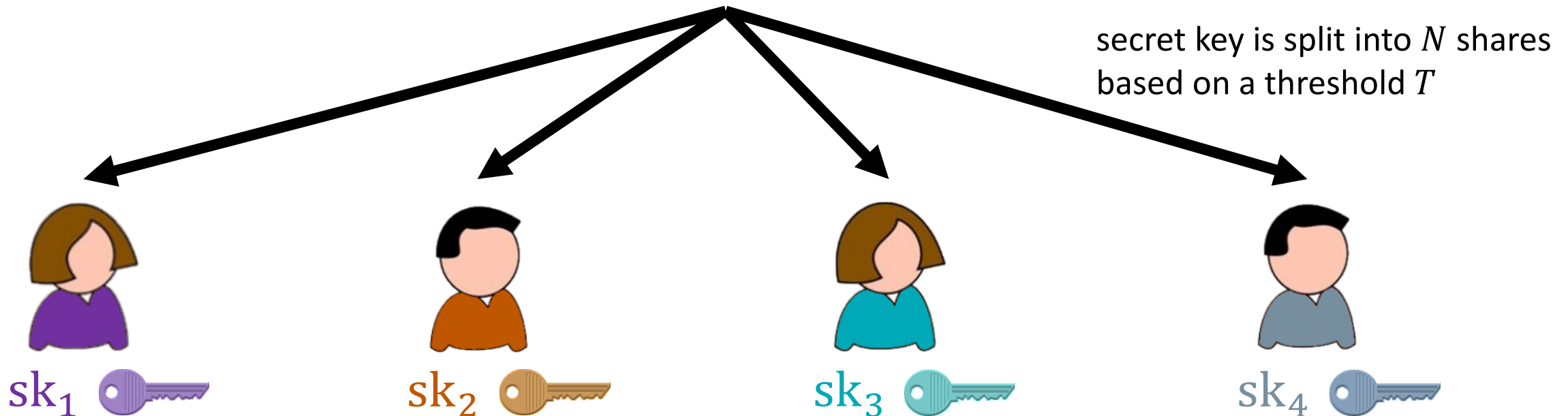
master public key



master secret key



dealer



Monotone-Policy Encryption

Functionality: Authorized sets of users can decrypt

Security: Unauthorized sets of users learn nothing

Efficiency: $|ct| = \text{poly}(\lambda, \log|P|)$



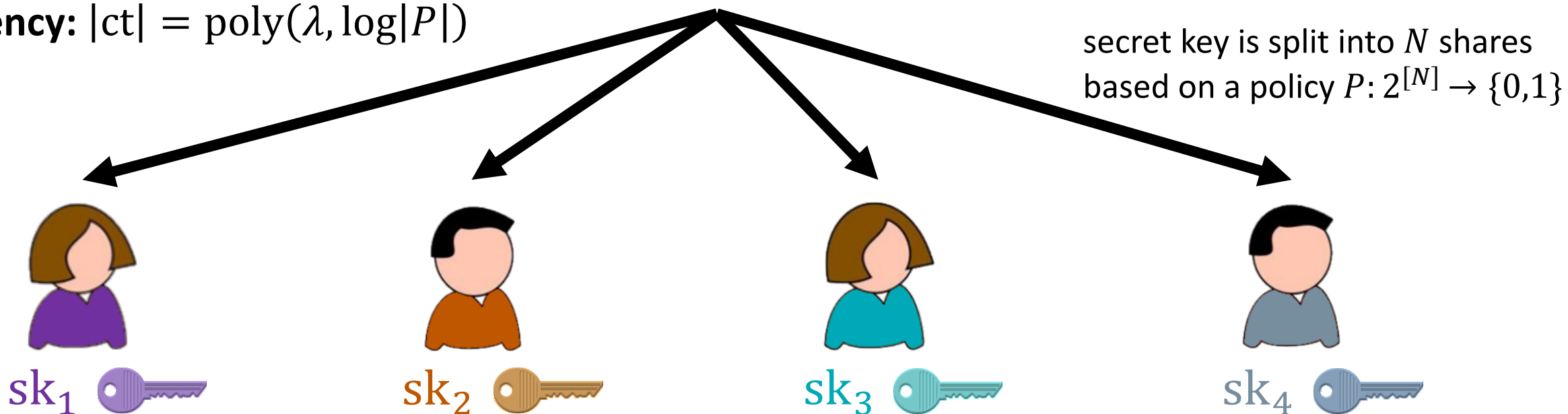
master public key



master secret key



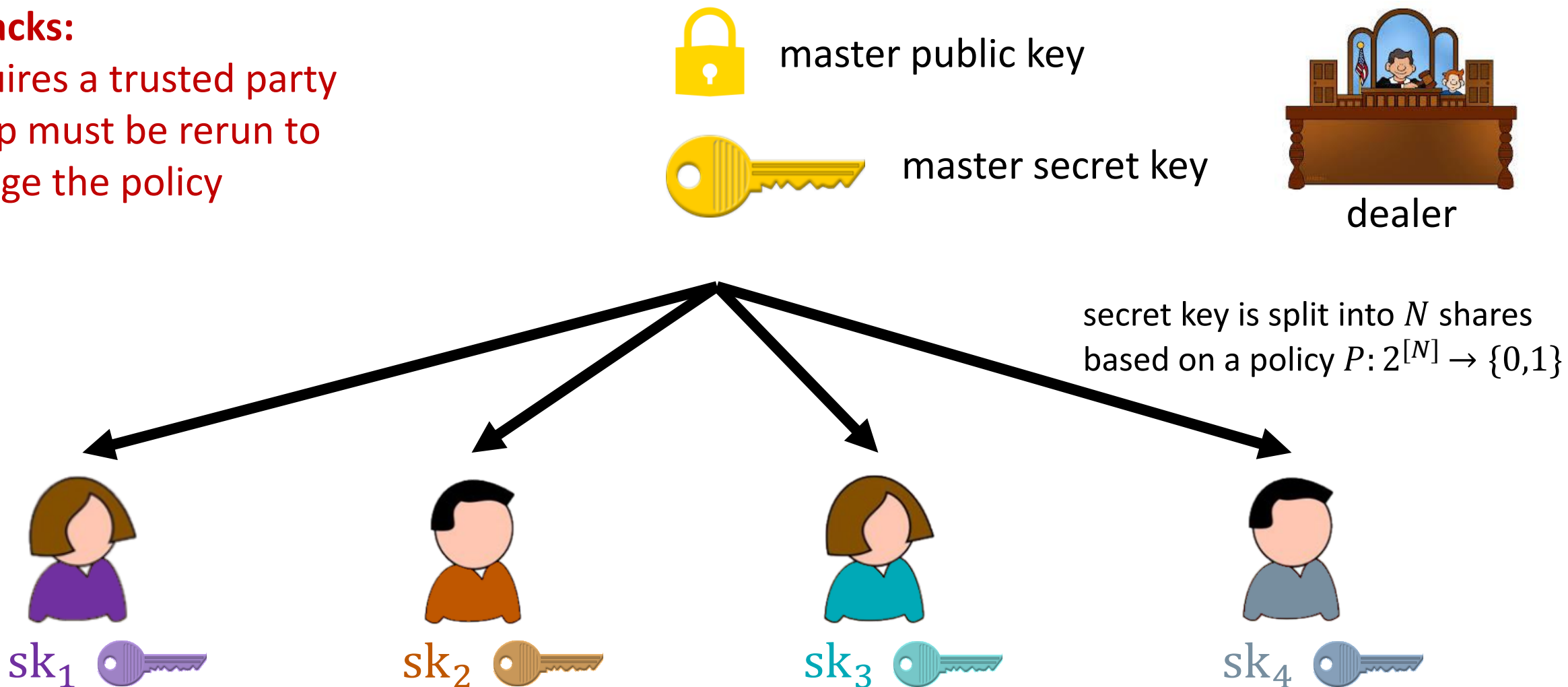
dealer



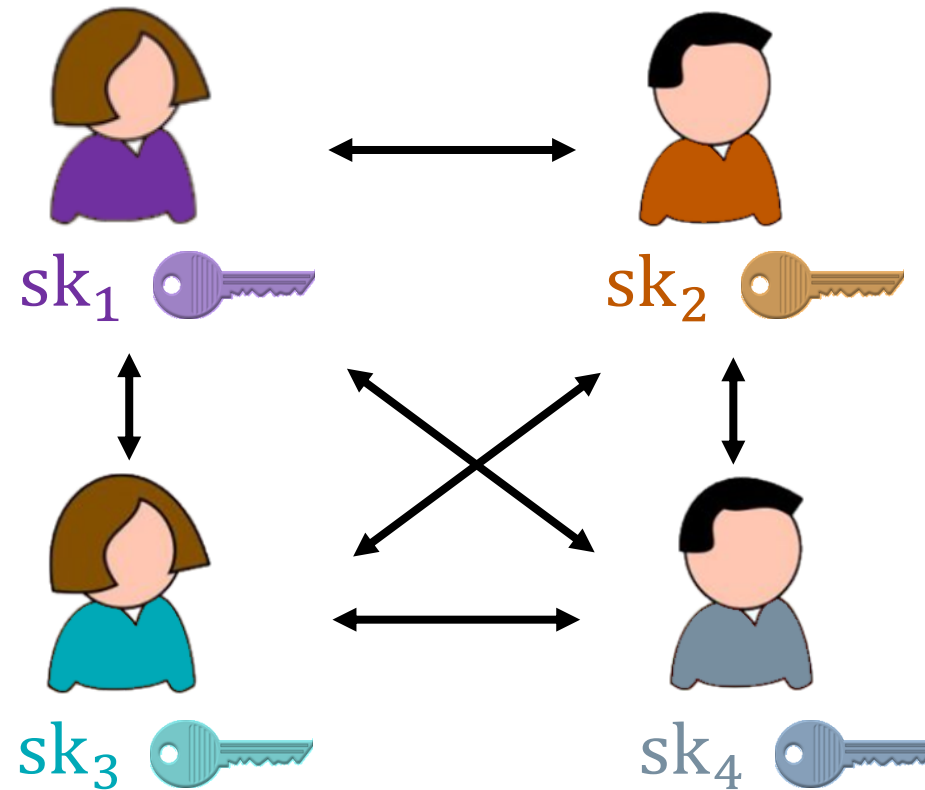
Monotone-Policy Encryption

Drawbacks:

- Requires a trusted party
- Setup must be rerun to change the policy

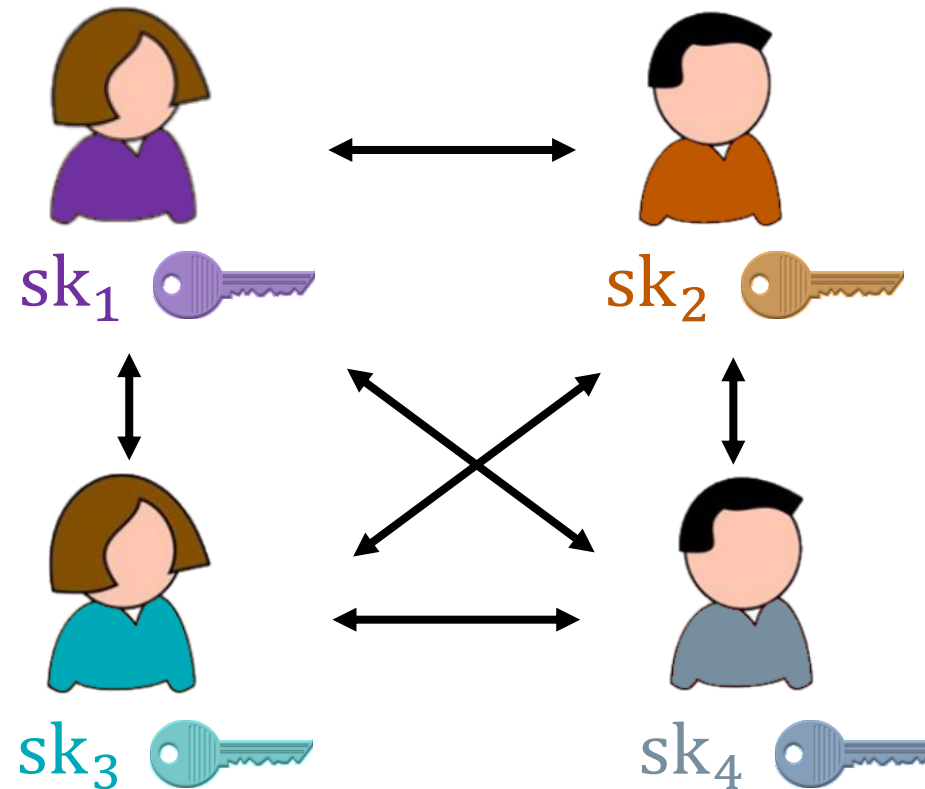


What if there is no trusted dealer?



Users can perform a distributed key generation protocol to get key shares with respect to the policy P

What if there is no trusted dealer?

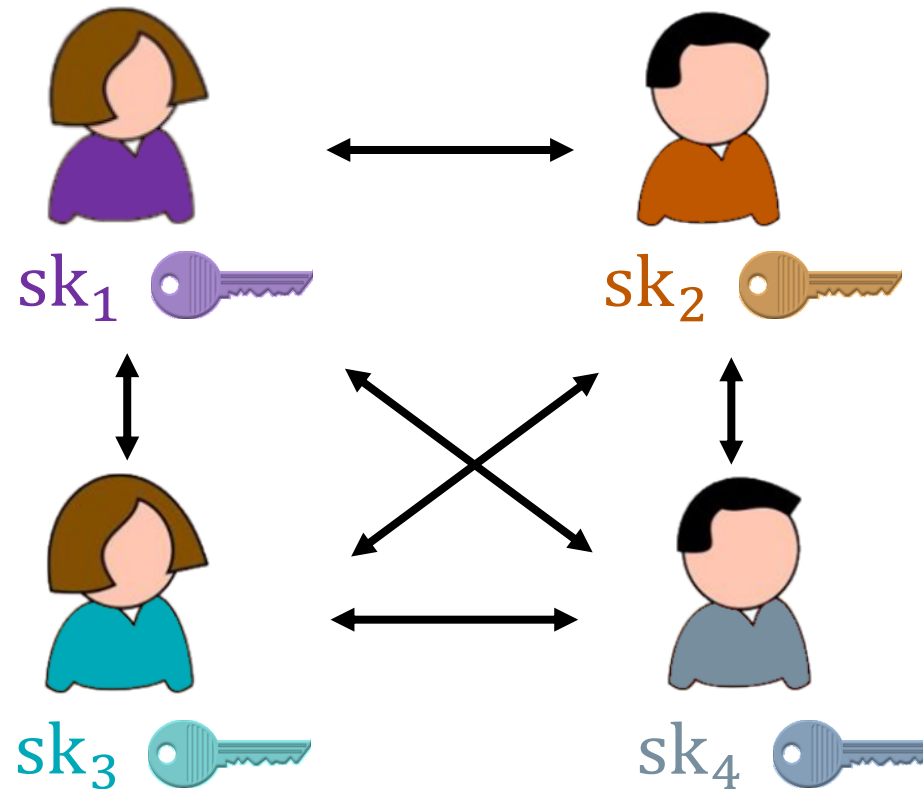


Users can perform a distributed key generation protocol to get key shares with respect to the policy P

Drawbacks:

- Users need to coordinate and interact
- Setup must be rerun to change the policy

What if there is no trusted dealer?



Users can perform a distributed key generation protocol to get key shares with respect to the policy P

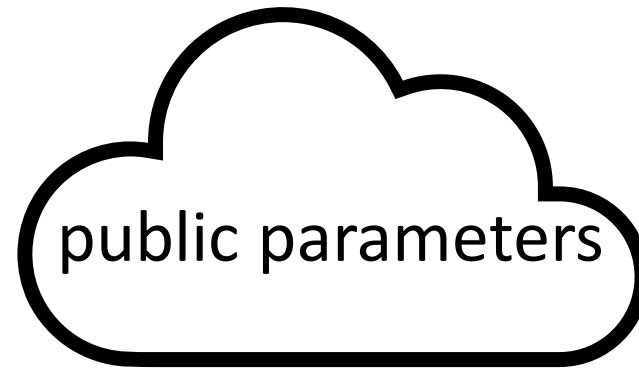
Drawbacks:

- Users need to coordinate and interact
- Setup must be rerun to change the policy

Can we hope for a trustless non-interactive setup and dynamic policies?

Distributed Monotone-Policy Encryption

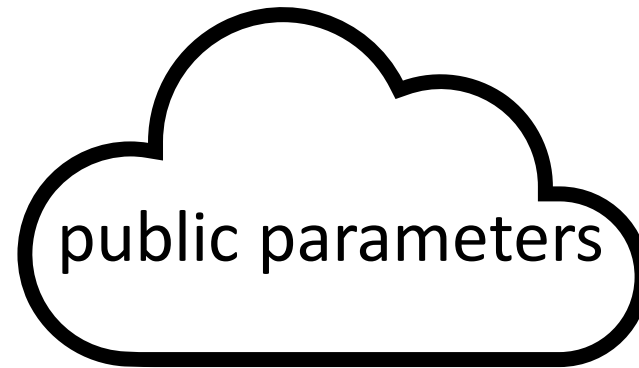
[ReyzinSmithYakubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



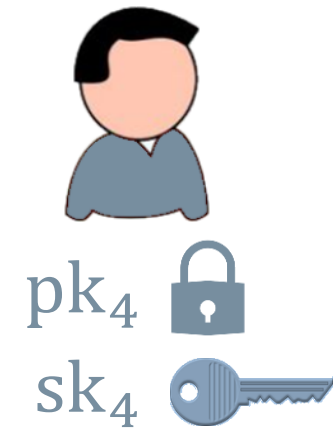
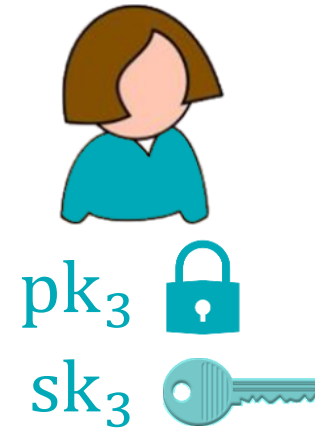
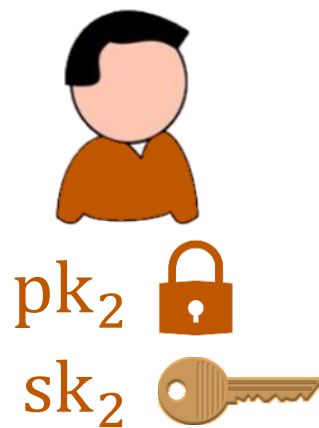
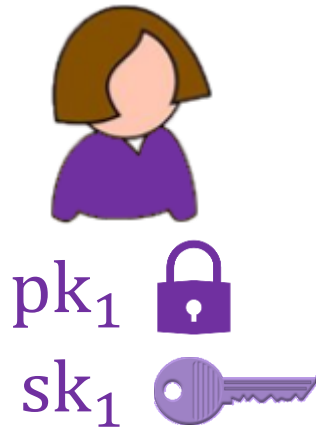
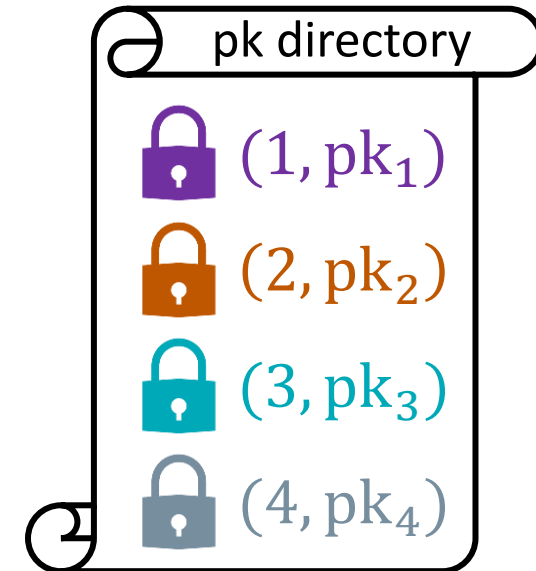
generated once and for all, no
long-term secrets stored

Distributed Monotone-Policy Encryption

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



generated once and for all, no
long-term secrets stored



Distributed Monotone-Policy Encryption

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]

message m



$$P = (1 \wedge 3 \wedge 4) \vee 2$$

ciphertext specifies
a set of public keys
and a policy P

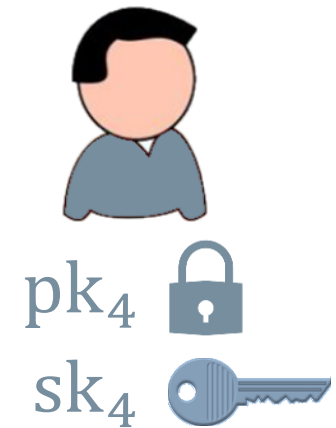
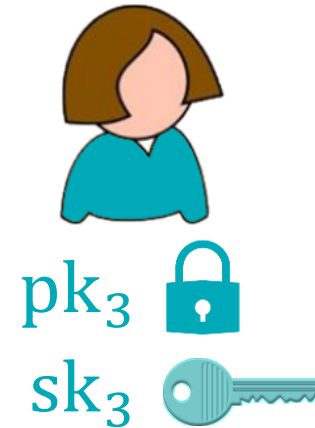
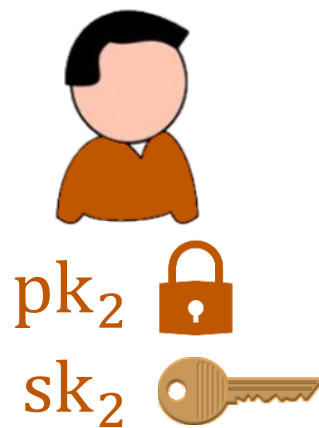
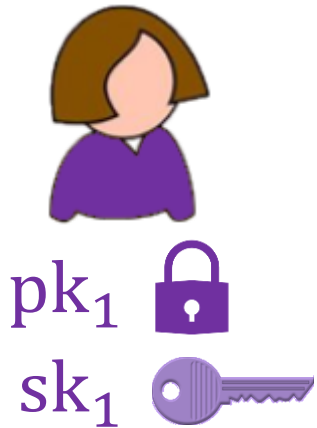
pk directory

 $(1, pk_1)$

 $(2, pk_2)$

 $(3, pk_3)$

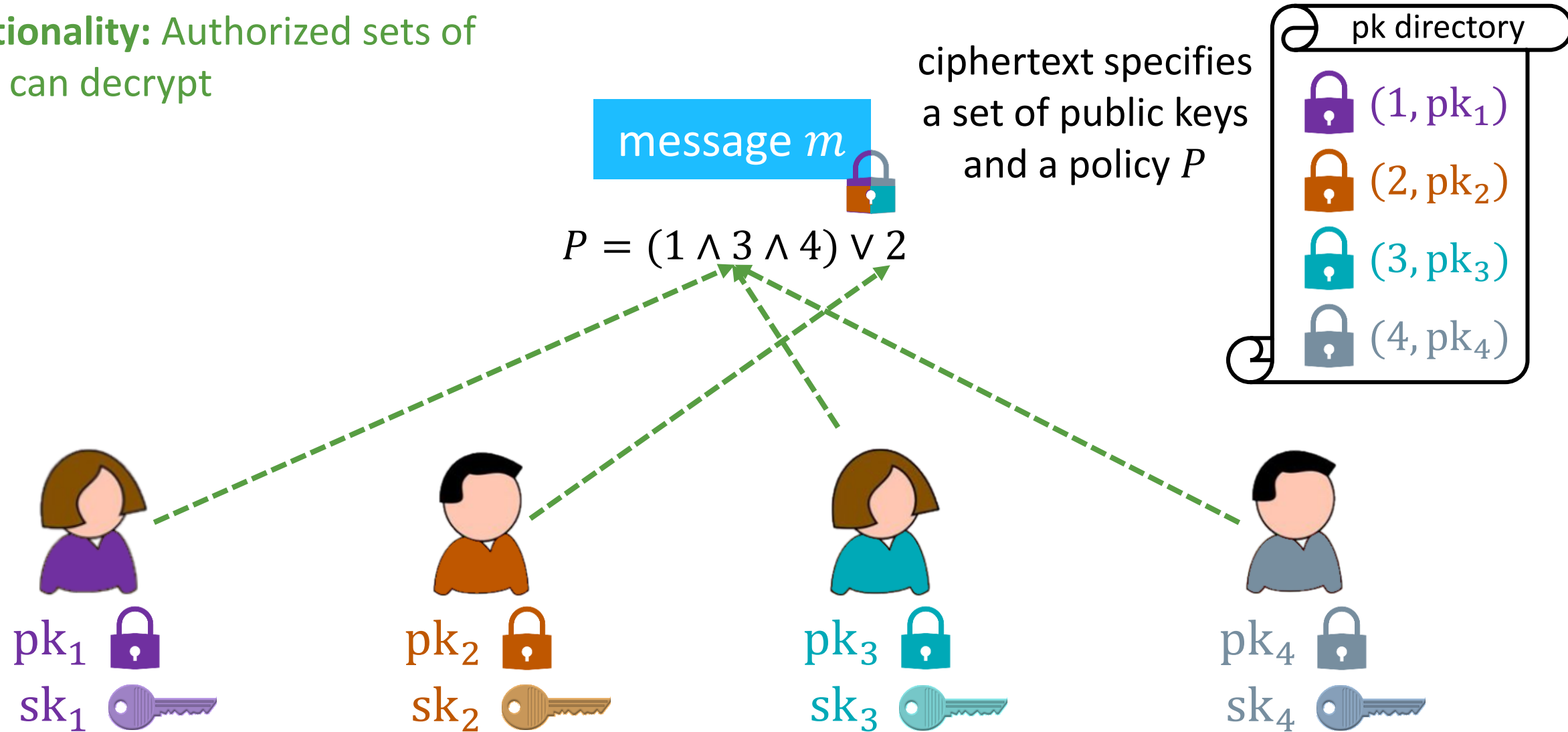
 $(4, pk_4)$



Distributed Monotone-Policy Encryption

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]

Functionality: Authorized sets of users can decrypt



Distributed Monotone-Policy Encryption

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]

Functionality: Authorized sets of users can decrypt

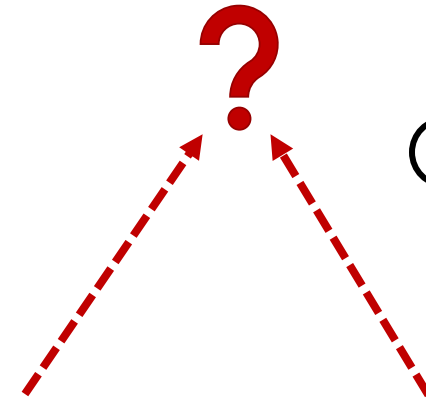
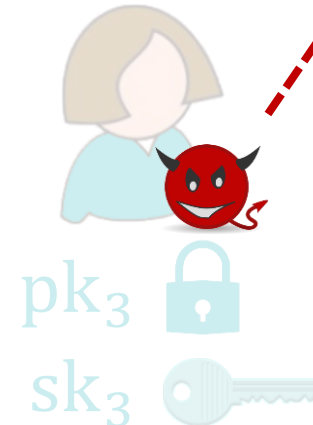
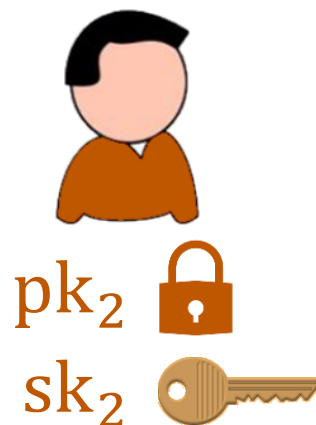
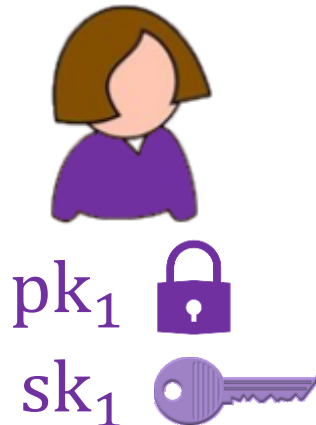
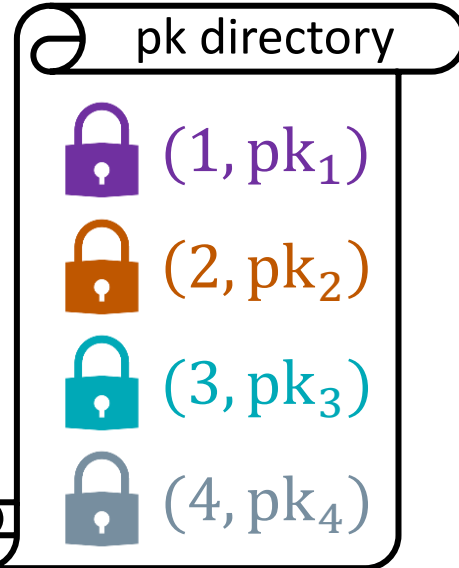
Security: Unauthorized sets of users learn nothing

message m



$$P = (1 \wedge 3 \wedge 4) \vee 2$$

ciphertext specifies
a set of public keys
and a policy P



Distributed Monotone-Policy Encryption

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]

Functionality: Authorized sets of users can decrypt

Security: Unauthorized sets of users learn nothing

Efficiency: $|ct| = \text{poly}(\lambda, \log|P|)$

message m



$$P = (1 \wedge 3 \wedge 4) \vee 2$$

ciphertext specifies
a set of public keys
and a policy P

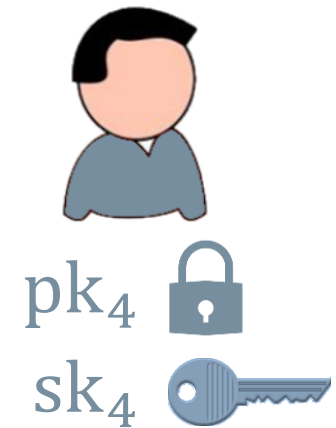
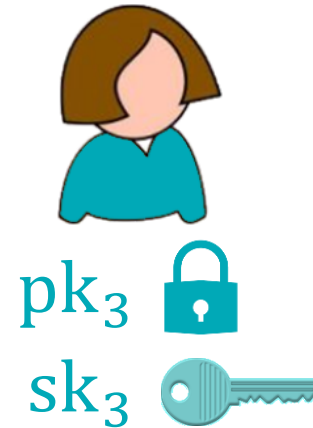
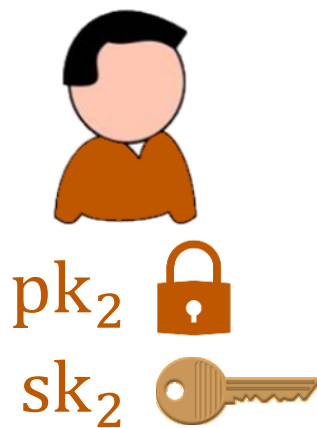
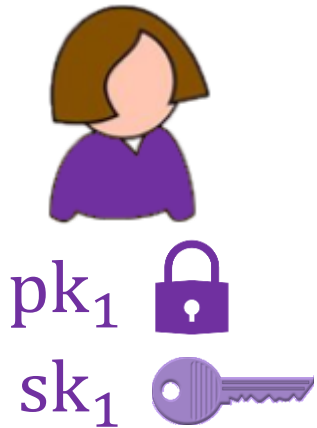
pk directory

 $(1, pk_1)$

 $(2, pk_2)$

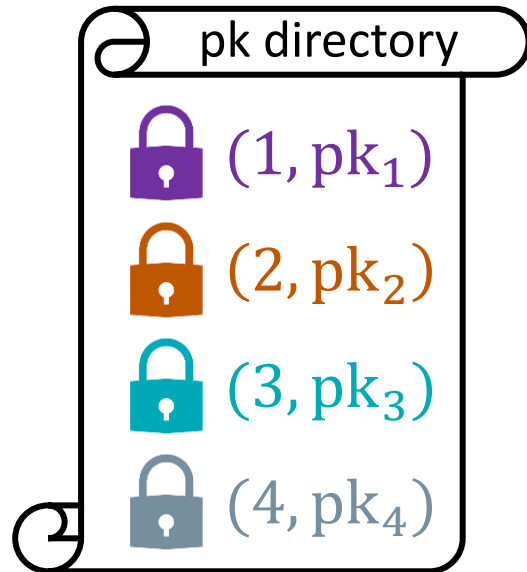
 $(3, pk_3)$

 $(4, pk_4)$



Distributed Monotone-Policy Encryption (DMPE)

[ReyzinSmithYakubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



$\text{KeyGen}(\text{pp}, i) \rightarrow (pk_i, sk_i)$

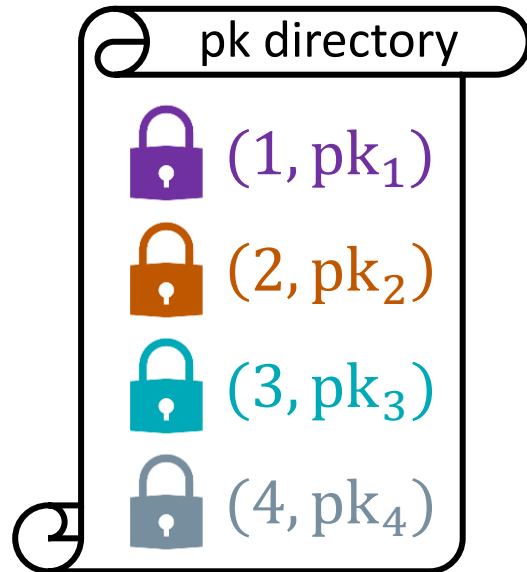
size $\text{poly}(\lambda, \log |P|)$

$\text{Encrypt}(\text{pp}, \{pk_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$

$P: 2^{[N]} \rightarrow \{0,1\}$

Distributed Monotone-Policy Encryption (DMPE)

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



$\text{KeyGen}(\text{pp}, i) \rightarrow (\text{pk}_i, \text{sk}_i)$

size $\text{poly}(\lambda, \log |P|)$

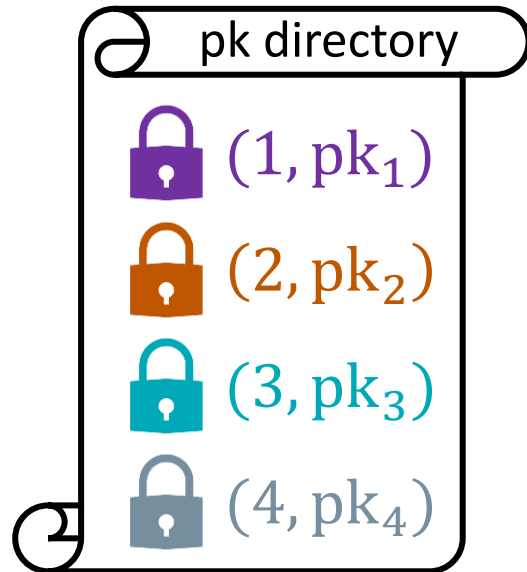
$\text{Encrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$

$P: 2^{[N]} \rightarrow \{0,1\}$

How should decryption work?

Distributed Monotone-Policy Encryption (DMPE)

[ReyzinSmithYakubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



$\text{KeyGen}(\text{pp}, i) \rightarrow (pk_i, sk_i)$

size $\text{poly}(\lambda, \log |P|)$

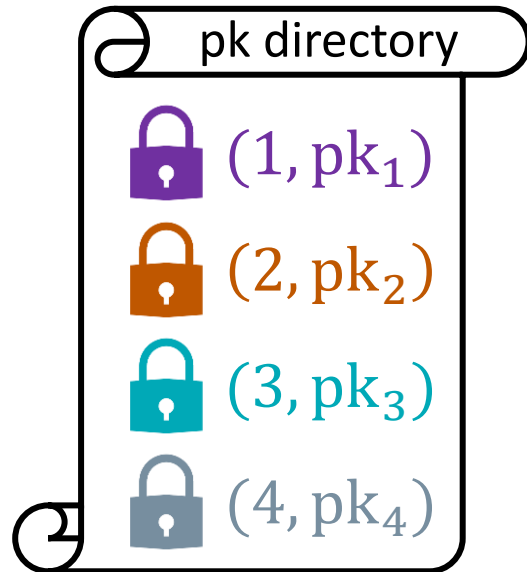
$\text{Encrypt}(\text{pp}, \{pk_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$

$P: 2^{[N]} \rightarrow \{0,1\}$

$\text{GetHint}(\text{pp}, sk_i, \text{ct}) \rightarrow \text{ht}_i$

Distributed Monotone-Policy Encryption (DMPE)

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



$\text{KeyGen}(\text{pp}, i) \rightarrow (\text{pk}_i, \text{sk}_i)$

size $\text{poly}(\lambda, \log |P|)$

$\text{Encrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$

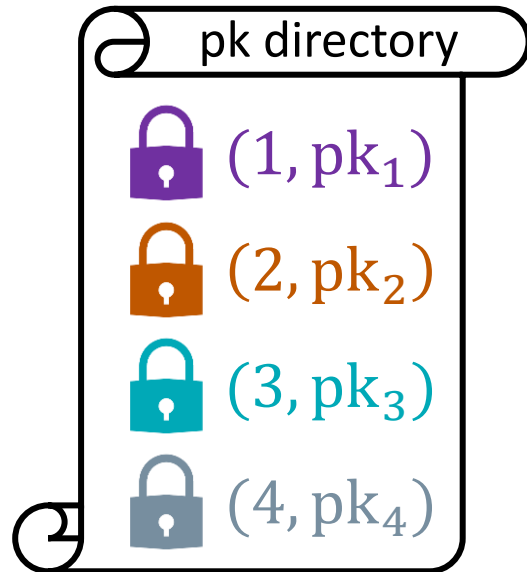
$P: 2^{[N]} \rightarrow \{0,1\}$

$\text{GetHint}(\text{pp}, \text{sk}_i, \text{ct}) \rightarrow \text{ht}_i$

$\text{Decrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, \{\text{ht}_i\}_{i \in T}, P, \text{ct}) \rightarrow m$

Distributed Monotone-Policy Encryption (DMPE)

[ReyzinSmithYakoubov21, GargKolonelosPolicharlaWang24, DevadasJainWatersWu25]



Functionality: Users in T can recover m if $P(T) = 1$

Security: Unqualified set of users learn nothing about m even given hints for other ciphertexts

$$\text{KeyGen}(\text{pp}, i) \rightarrow (\text{pk}_i, \text{sk}_i)$$

size $\text{poly}(\lambda, \log |P|)$

$$\text{Encrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$$

$P: 2^{[N]} \rightarrow \{0,1\}$

$$\text{GetHint}(\text{pp}, \text{sk}_i, \text{ct}) \rightarrow \text{ht}_i$$

$$\text{Decrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, \{\text{ht}_i\}_{i \in T}, P, \text{ct}) \rightarrow m$$

Another View: Reusable Succinct Secret Sharing

“reusable share distribution”

$\text{KeyGen}(\text{pp}, i) \rightarrow (\text{pk}_i, \text{sk}_i)$

size $\text{poly}(\lambda, \log |P|)$

“succinct sharing of m ”

$\text{Encrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, P, m) \rightarrow \text{ct}$

$P: 2^{[N]} \rightarrow \{0,1\}$

“local share reconstruction”

$\text{GetHint}(\text{pp}, \text{sk}_i, \text{ct}) \rightarrow \text{ht}_i$

“secret recovery”

$\text{Decrypt}(\text{pp}, \{\text{pk}_i\}_{i \in [N]}, \{\text{ht}_i\}_{i \in T}, P, \text{ct}) \rightarrow m$

Prior Work on DMPE

- iO: read-once bounded-space TMs [DJWW25]
- Witness Encryption: CNFs/DNFs [DJWW25]
- Pairings: thresholds [GKPW24], Boolean formulas [WW25]
- Lattices: disjunctions [CW24, WW25]

Prior Work on DMPE

- iO: read-once bounded-space TMs [DJWW25] impractical + strong assumption
- Witness Encryption: CNFs/DNFs [DJWW25] impractical + strong assumption
- Pairings: thresholds [GKPW24], Boolean formulas [WW25] no security against quantum attacks
- Lattices: disjunctions [CW24, WW25] single-user decryption

Prior Work on DMPE

- iO: read-once bounded-space TMs [DJWW25] impractical + strong assumption
- Witness Encryption: CNFs/DNFs [DJWW25] impractical + strong assumption
- Pairings: thresholds [GKPW24], Boolean formulas [WW25] no security against quantum attacks
- Lattices: disjunctions [CW24, WW25] single-user decryption

Constructions for interesting policies from lattice assumptions?

Our Results

DMPE supporting DNF formulas for unbounded users from decomposed LWE in the random oracle model

- **Public parameter size:** $O(1)$ transparent setup
- **User public key size:** $O(k)$, where k bounds the # of times this variable can appear
- **User secret key size:** $O(1)$
- **Ciphertext size:** $O(1)$

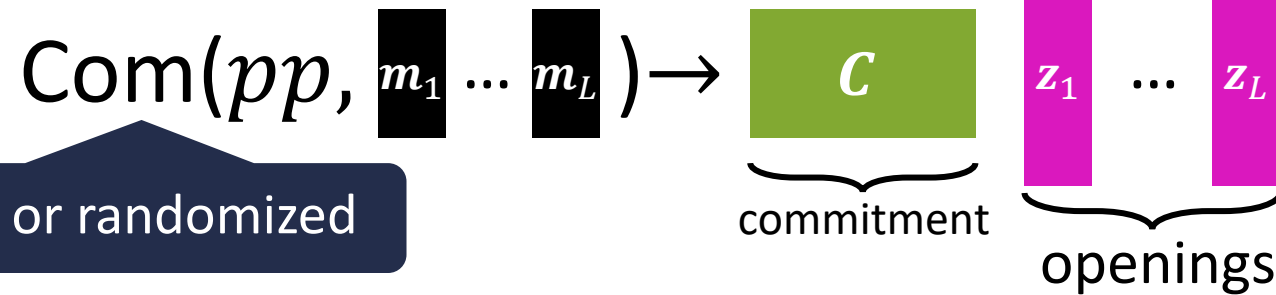
First scheme for DNFs and unbounded users without iO/WE!

$\text{poly}(\lambda, \log N)$ factors omitted

Matrix Commitments

[Wee25, adapted]

“vector commitment to vectors”

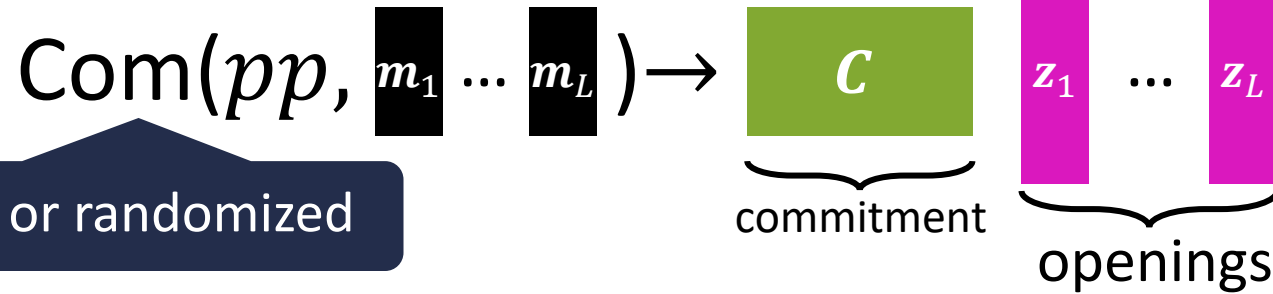


can be deterministic or randomized

Matrix Commitments

[Wee25, adapted]

“vector commitment to vectors”



can be deterministic or randomized

For all $i \in [L]$:

$$\underbrace{C}_{\text{commitment}} v_i = m_i - A z_i$$

low-norm

pp describes A
and v_i for $i \in [L]$

Matrix Commitments

[Wee25, adapted]

“vector commitment to vectors”

$$\text{Com}(pp, \mathbf{m}_1 \dots \mathbf{m}_L) \rightarrow \underbrace{\mathbf{C}}_{\text{commitment}} \underbrace{\mathbf{z}_1 \dots \mathbf{z}_L}_{\text{openings}}$$

can be deterministic or randomized

For all $i \in [L]$:

$$\mathbf{C} \mathbf{v}_i = \mathbf{m}_i - \mathbf{A} \mathbf{z}_i$$

low-norm

pp describes $\mathbf{A}$
and $\mathbf{v}_i$ for $i \in [L]$

Security: $\mathbf{s}^T \mathbf{A} + \mathbf{e}^T \approx \mathbf{z}^T$ given pp

Matrix Commitments

[Wee25, adapted]

“vector commitment to vectors”

$$\text{Com}(pp, \mathbf{m}_1 \dots \mathbf{m}_L) \rightarrow \underbrace{\mathcal{C}}_{\text{commitment}} \underbrace{\mathbf{z}_1 \dots \mathbf{z}_L}_{\text{openings}}$$

can be deterministic or randomized

For all $i \in [L]$:

$$\mathcal{C} \quad \mathbf{v}_i = \mathbf{m}_i - \mathbf{A} \mathbf{z}_i$$

low-norm

pp describes $\mathbf{A}$
and $\mathbf{v}_i$ for $i \in [L]$

Can instantiate from succinct LWE [Wee24]
or decomposed LWE [AMR25]

Security: $\mathbf{s}^T \mathbf{A} + \mathbf{e}^T \approx \mathbf{z}^T$ given pp

Starting Point: DMPE for Disjunction [WW25]

Starting Point: DMPE for Disjunction [WW25]

- Each user will sample a (dual-Regev) key pair and publish the public key

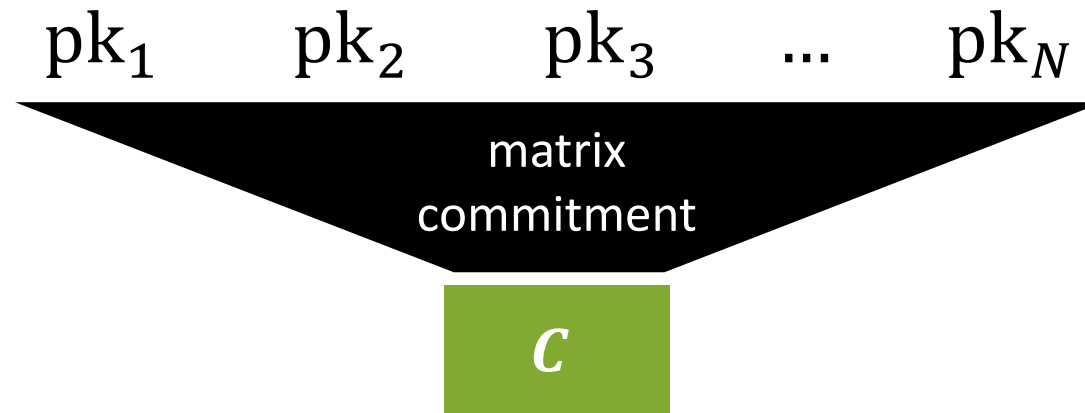
Starting Point: DMPE for Disjunction [WW25]

- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:

$pk_1 \quad pk_2 \quad pk_3 \quad \dots \quad pk_N$

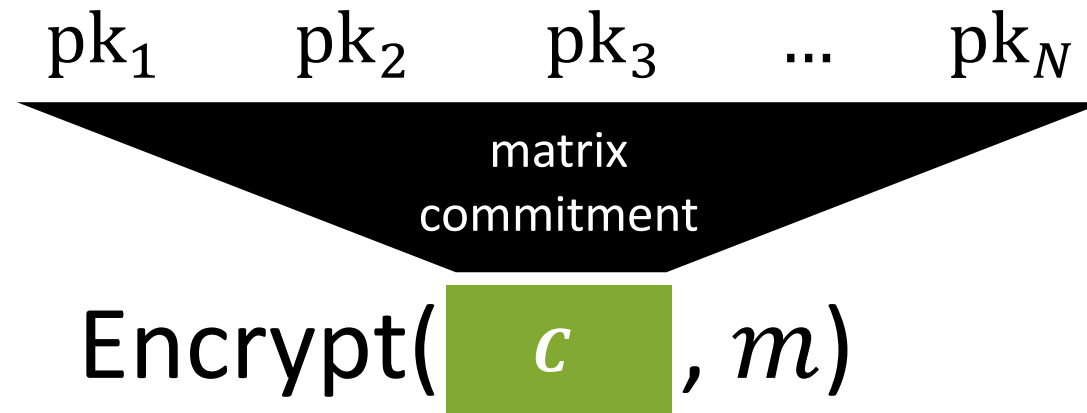
Starting Point: DMPE for Disjunction [WW25]

- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:



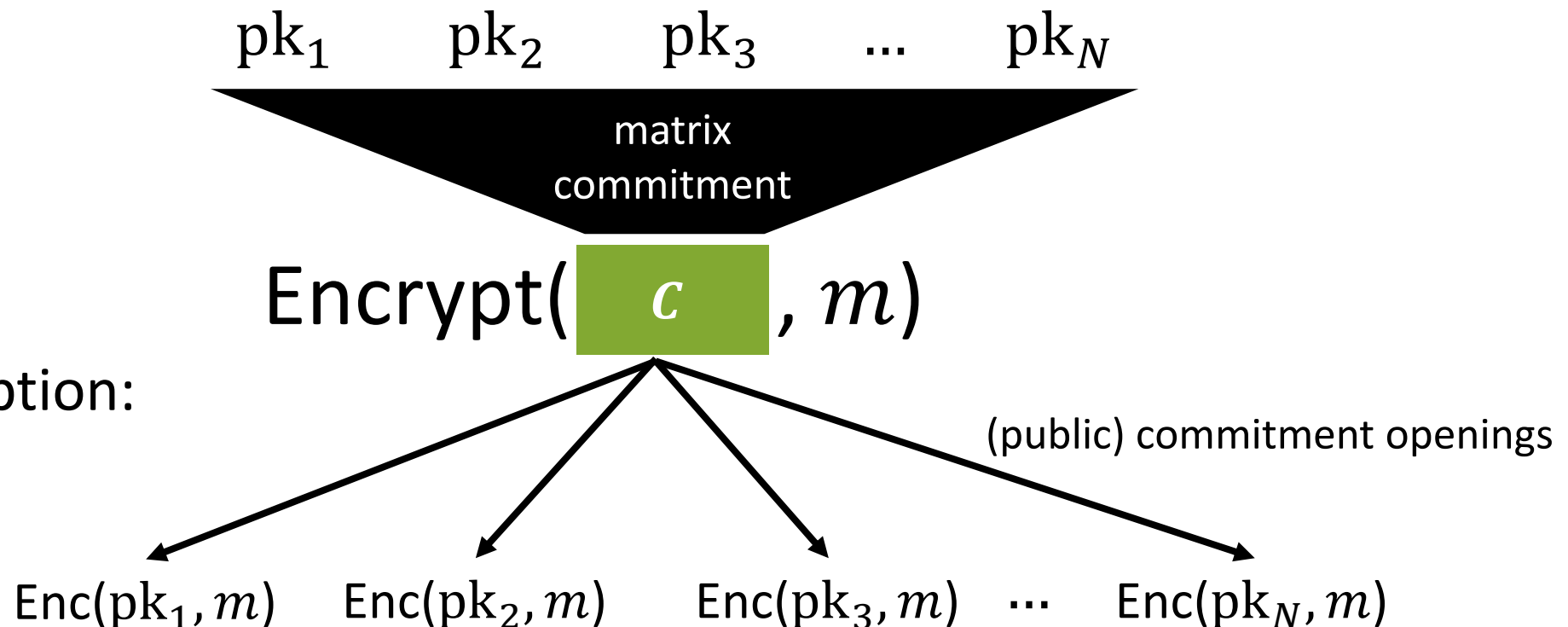
Starting Point: DMPE for Disjunction [WW25]

- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:



Starting Point: DMPE for Disjunction [WW25]

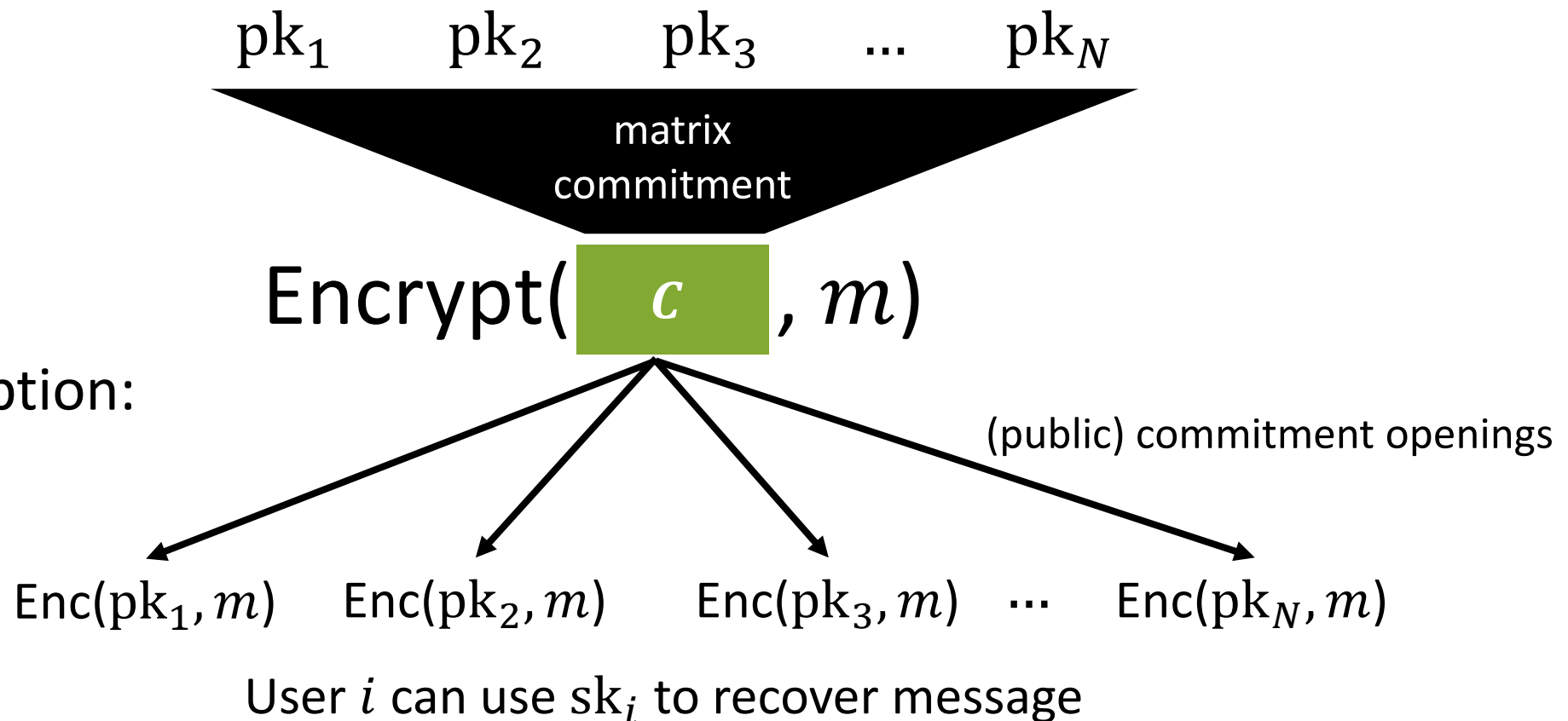
- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:



- Decryption:

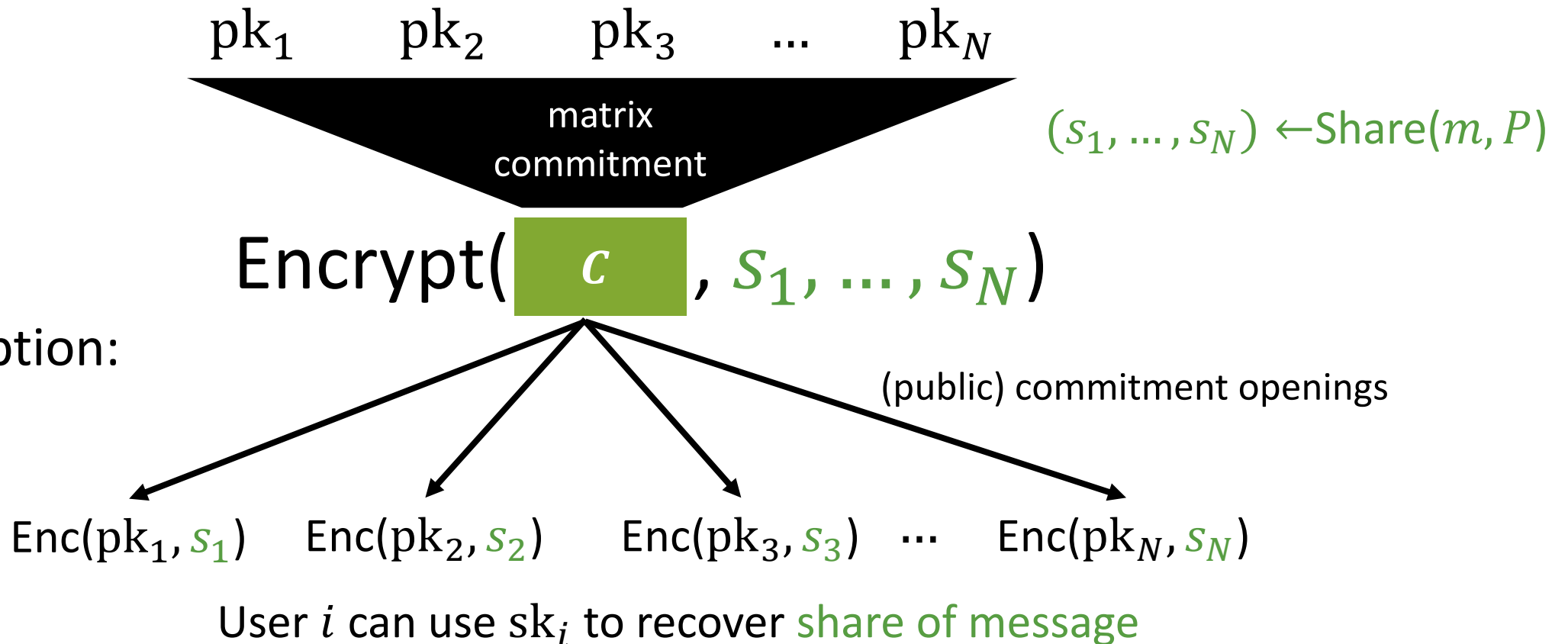
Starting Point: DMPE for Disjunction [WW25]

- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:

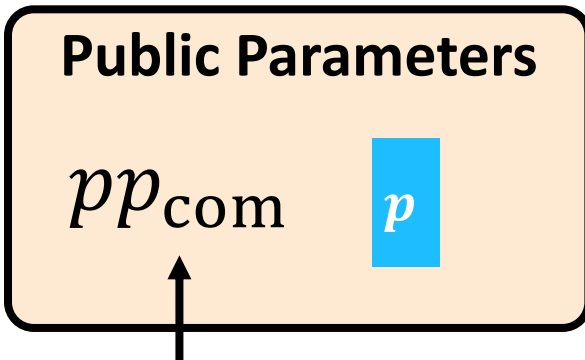


This Work: Extending to Boolean Formulas

- Each user will sample a (dual-Regev) key pair and publish the public key
- Encryption does the following:



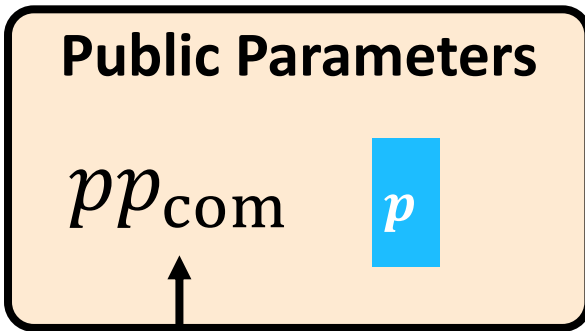
Setup and Key Generation



includes A, v_i

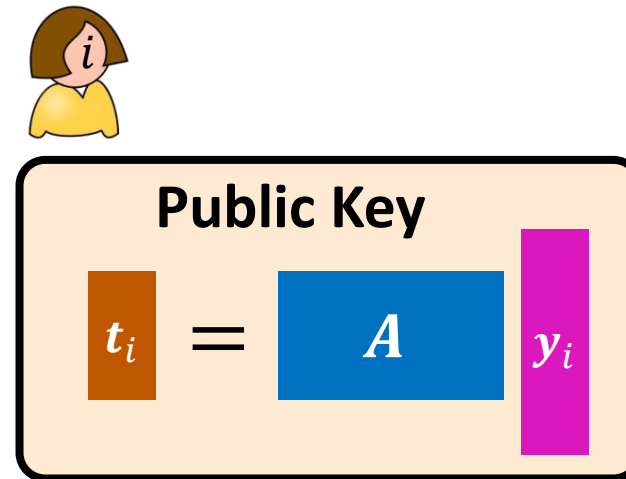
deterministic commitment
[Wee25] has fixed-size
public parameters!

Setup and Key Generation



includes A, v_i

deterministic commitment
[Wee25] has fixed-size
public parameters!



low-norm secret key

Encryption

Public Parameters

Given

pp_{com} (includes A, v_i) p

Public Keys

t_1 ... t_N

Encryption of $b \in \{0,1\}$ to P :

Encryption

Public Parameters

Given

pp_{com} (includes A, v_i) p

Public Keys

$t_1 \dots t_N$

Encryption of $b \in \{0,1\}$ to P :

s^T p
└───┘
masks b

Noise omitted

Encryption

Public Parameters

Given

pp_{com} (includes A, v_i) p

Public Keys

$t_1 \dots t_N$

Encryption of $b \in \{0,1\}$ to P :

Idea: share p with respect to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$

Actual secret is $s^T p$ and share j is $s^T \hat{p}_j$, public shares denote structure and are derived with *fresh* share randomness

s^T p
└───┬───┘
masks b

Noise omitted

Encryption

Public Parameters

Given

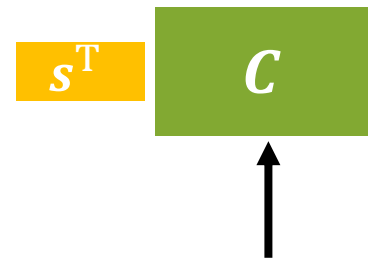
pp_{com} (includes A , v_i) p

Public Keys

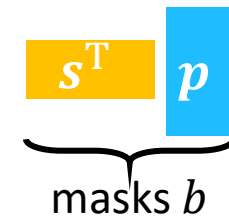
t_1 ... t_N

Encryption of $b \in \{0,1\}$ to P :

Idea: share p with respect to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$



commit to $\hat{p}_j + t_j$ at position $j \in [N]$



Noise omitted

Encryption

Public Parameters

Given

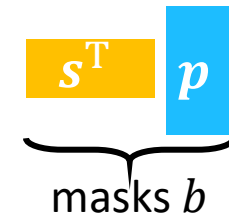
pp_{com} (includes A , v_i) p

Public Keys

t_1 ... t_N

Encryption of $b \in \{0,1\}$ to P :

Idea: share p with respect to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$



commit to $\hat{p}_j + t_j$ at position $j \in [N]$

Noise omitted

Encryption

Public Parameters

Given

pp_{com} (includes A , v_i) p

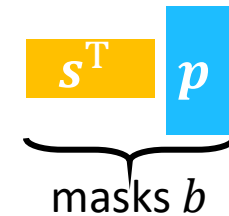
Public Keys

t_1 ... t_N

Encryption of $b \in \{0,1\}$ to P :

Idea: share p with respect to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$

Issue: how to succinctly generate the public shares?



commit to $\hat{p}_j + t_j$ at position $j \in [N]$

Noise omitted

Encryption

Public Parameters

Given

pp_{com} (includes A , v_i) p

Public Keys

t_1 ... t_N

Encryption of $b \in \{0,1\}$ to P :

Idea: share p with respect to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$

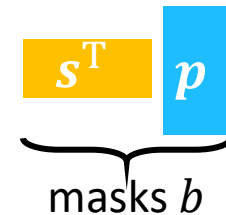
Issue: how to succinctly generate the public shares?



random oracle seed



generates public shares



commit to $\hat{p}_j + t_j$ at position $j \in [N]$

Noise omitted

Encryption

Given **Public Parameters** **Public Keys**

pp_{com} (includes A , v_i) p $t_1 \dots t_N$

Encryption of $b \in \{0,1\}$ to P :

if P is a disjunction, there is no share randomness, so random oracle seed can be dropped to get [WW25] to P to get *public* shares $\hat{p}_1, \dots, \hat{p}_N$
generate the public shares?



Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

For all $i \in [N]$: $Cv_i = \hat{p}_i + t_i - Az_i$

public and low-norm

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

For all $i \in [N]$: $Cv_i = \hat{p}_i + t_i - Az_i$

$$Cv_i + Az_i = \hat{p}_i + t_i$$

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

For all $i \in [N]$: $Cv_i = \hat{p}_i + t_i - Az_i$

$$Cv_i + Az_i = \hat{p}_i + t_i$$

$s^T C$

$s^T A$

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

For all $i \in [N]$: $Cv_i = \hat{p}_i + t_i - Az_i$

$$Cv_i + Az_i = \hat{p}_i + t_i$$

$$s^T C v_i \quad s^T A z_i$$

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

For all $i \in [N]$: $Cv_i = \hat{p}_i + t_i - Az_i$

$$Cv_i + Az_i = \hat{p}_i + t_i$$

$$s^T Cv_i + s^T Az_i \approx s^T \hat{p}_i + s^T t_i$$

public

encryption of share under pk_i

Decryption

Public Parameters

pp_{com}

p

includes A, v_i

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T A

s^T C

Noise omitted

s^T p

masks b

only user i can compute

$$s^T t_i \approx s^T A y_i$$

$$\text{For all } i \in [N]: C v_i = \hat{p}_i + t_i - A z_i$$

$$C v_i + A z_i = \hat{p}_i + t_i$$

$$s^T C v_i + s^T A z_i \approx s^T \hat{p}_i + s^T t_i$$

public

encryption of share under pk_i

Public Key

t_i

=

A

y_i

low-norm secret key

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

Given noisy shares $s^T \hat{p}_i$ for $i \in T$ such that $P(T) = 1$, need a secret sharing to have a linear and low-norm reconstruction process to recover a noisy $s^T p$

Decryption

Public Parameters

pp_{com}

p

includes A, v_i



Public Key

t_i

=

A

y_i

low-norm secret key

Encryption of $b \in \{0,1\}$ to P :



random oracle seed

s^T

A

s^T

C

s^T

p

Noise omitted

masks b

Given noisy shares $s^T \hat{p}_i$ for $i \in T$ such that $P(T) = 1$, need a secret sharing to have a linear and low-norm reconstruction process to recover a noisy $s^T p$

LSSS with binary coefficients contains Boolean formulas and NC1 [BL88, LW11]

Why is Security Limited to DNFs?

Consider the policy $P = (1 \vee 2) \wedge 3$

Why is Security Limited to DNFs?

Consider the policy $P = (1 \vee 2) \wedge 3$

The explicit LSSS with binary coefficients computes a sharing of p as follows:

1. Samples p' randomly
2. Sets $\hat{p}_1 = \hat{p}_2 = p'$
3. Sets $\hat{p}_3 = p - p'$

Why is Security Limited to DNFs?

Consider the policy $P = (1 \vee 2) \wedge 3$

The explicit LSSS with binary coefficients computes a sharing of $\mathbf{p}$ as follows:

1. Samples $\mathbf{p}'$ randomly
2. Sets $\hat{\mathbf{p}}_1 = \hat{\mathbf{p}}_2 = \mathbf{p}'$
3. Sets $\hat{\mathbf{p}}_3 = \mathbf{p} - \mathbf{p}'$

If the adversary has shares 1 and 2, it can derive noisy versions of $\mathbf{s}^T \hat{\mathbf{p}}_1$ and $\mathbf{s}^T \hat{\mathbf{p}}_2$ with *different* error terms that depend on the secret keys of user 1 and 2

Why is Security Limited to DNFs?

Consider the policy $P = (1 \vee 2) \wedge 3$

The explicit LSSS with binary coefficients computes a sharing of $\mathbf{p}$ as follows:

1. Samples $\mathbf{p}'$ randomly
2. Sets $\hat{\mathbf{p}}_1 = \hat{\mathbf{p}}_2 = \mathbf{p}'$
3. Sets $\hat{\mathbf{p}}_3 = \mathbf{p} - \mathbf{p}'$

If the adversary has shares 1 and 2, it can derive noisy versions of $\mathbf{s}^T \hat{\mathbf{p}}_1$ and $\mathbf{s}^T \hat{\mathbf{p}}_2$ with *different* error terms that depend on the secret keys of user 1 and 2

Since $\mathbf{s}^T \hat{\mathbf{p}}_1 = \mathbf{s}^T \hat{\mathbf{p}}_2 = \mathbf{s}^T \mathbf{p}'$, subtracting these noisy shares **leaks the LWE error!**

Why is Security Limited to DNFs?

Consider the policy $P = (1 \vee 2) \wedge 3$

The explicit LSSS with binary coefficients computes a sharing of $\mathbf{p}$ as follows:

1. Samples $\mathbf{p}'$ randomly
2. Sets $\hat{\mathbf{p}}_1 = \hat{\mathbf{p}}_2 = \mathbf{p}'$
3. Sets $\hat{\mathbf{p}}_3 = \mathbf{p} - \mathbf{p}'$

Attack fails if unauthorized shares $\hat{\mathbf{p}}_i$ are **linearly independent** of each other, which holds for DNFs [DKW21]

If the adversary has shares 1 and 2, it can derive noisy versions of $\mathbf{s}^T \hat{\mathbf{p}}_1$ and $\mathbf{s}^T \hat{\mathbf{p}}_2$ with *different* error terms that depend on the secret keys of user 1 and 2

Since $\mathbf{s}^T \hat{\mathbf{p}}_1 = \mathbf{s}^T \hat{\mathbf{p}}_2 = \mathbf{s}^T \mathbf{p}'$, subtracting these noisy shares **leaks the LWE error!**

Generating Hints Safely

For all $i \in [N]$: $\mathbf{C}\mathbf{v}_i = \hat{\mathbf{p}}_i + \mathbf{t}_i - \mathbf{A}\mathbf{z}_i$

$$\mathbf{C}\mathbf{v}_i + \mathbf{A}\mathbf{z}_i = \hat{\mathbf{p}}_i + \mathbf{t}_i$$

$$\underbrace{s^T \mathbf{C}\mathbf{v}_i + s^T \mathbf{A}\mathbf{z}_i}_{\text{public}} \approx \underbrace{s^T \hat{\mathbf{p}}_i + s^T \mathbf{t}_i}_{\text{encryption of share under } \text{pk}_i}$$

only $s^T \mathbf{t}_i \approx s^T \mathbf{A}\mathbf{y}_i$
needs a user's key

Generating Hints Safely

$$\text{For all } i \in [N]: \mathbf{C}\mathbf{v}_i = \hat{\mathbf{p}}_i + \mathbf{t}_i - \mathbf{A}\mathbf{z}_i$$

$$\mathbf{C}\mathbf{v}_i + \mathbf{A}\mathbf{z}_i = \hat{\mathbf{p}}_i + \mathbf{t}_i$$

$$\underbrace{s^T \mathbf{C}\mathbf{v}_i + s^T \mathbf{A}\mathbf{z}_i}_{\text{public}} \approx \underbrace{s^T \hat{\mathbf{p}}_i + s^T \mathbf{t}_i}_{\text{encryption of share under } \text{pk}_i}$$

only $s^T \mathbf{t}_i \approx s^T \mathbf{A}\mathbf{y}_i$
needs a user's key

Can give out $s^T \mathbf{A}\mathbf{y}_i$ by adding a large smudging error to remove the dependence on the key and adding a NIZK to the ciphertext to prove $s^T \mathbf{A}$ is well-formed

Open Problems

- Efficient and secure construction for CNFs? Thresholds? Boolean formulas? Can we get succinct secret sharing for these policies?
- Can we get a scheme with batch decryption?
- A compiler to handle corruptions? (e.g., something like GentryWaters09)

Thanks for listening!

<https://eprint.iacr.org/2026/318>